Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Hybrid Decoupled Storage Engine

Student / Team Information

Team Name: *Team ID:*	*Code Committers* *OSDBMS-V-2026-T044*
Team member 1 (Team Lead) *(Last Name, Name – Student ID – Email – Picture):*	*Singh, Vignesh – 24021761* *24021761@geu.ac.in* 
Team member 2 *(Last Name, Name – Student ID – Email – Picture):*	*Malasi, Satvik – 240211353* *240211353@geu.ac.in* 
Team member 3 *(Last Name, Name – Student ID – Email – Picture):*	*Singh, Manoj – 240211600* *240211600@geu.ac.in* 
Team member 4 *(Last Name, Name – Student ID – Email – Picture):*	*Bohra, Moulik – 24021011* *24021011@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

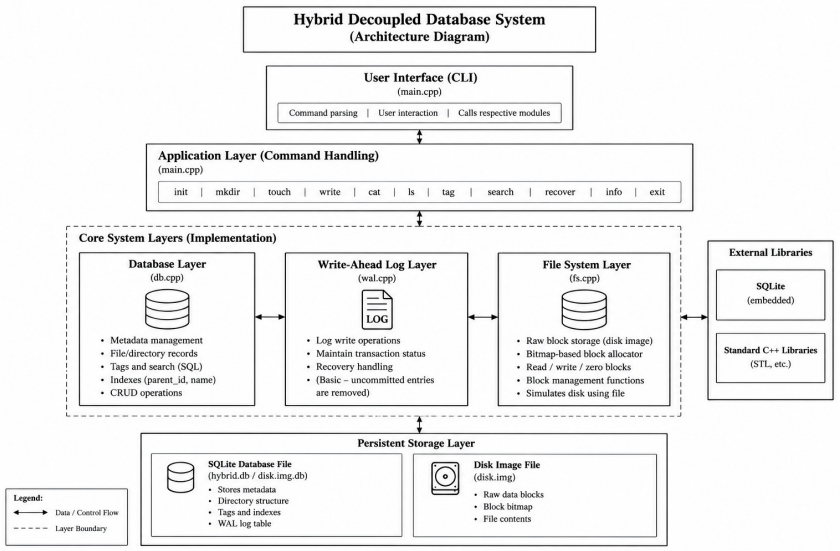
Project Abstract (2 pts)

Brief restatement of the project's main goal.

Hybrid Decoupled System is a user-space prototype that separates file metadata from file content, as S3 and HDFS do. Metadata (names, hierarchy, owner, mode, size, timestamps, custom tags and block pointers) lives in an SQLite database. File bytes live in a raw disk image as fixed 1,024-byte blocks, allocated with an EXT2-style bitmap kept in block 0. A write-ahead log table (wal_log) records each block write before it reaches the disk and marks it committed afterwards. A C command-line interface ties the layers together. Working commands: init, mkdir, touch, write, cat, ls, tag, search, recover and info. The core create/read/list/tag/search flow works. Deletion and full crash recovery are still pending.

Updated Project Approach and Architecture (2 pts)

Describe the current approach, system design, communication protocols, and libraries.



Layers

- Data layer (fs/fs.c) does raw block read/write and bitmap allocation. It knows nothing about files.
- Metadata layer (db/db.c) holds a files table with indexes on parent_id and name. It handles CRUD, path resolution (walking /a/b one component at a time) and tag search.
- Consistency layer (wal/wal.c) provides wal_log_block_write, wal_commit and wal_recover.
- CLI (cli/main.c) orchestrates the other three.

Write flow: resolve path, read metadata, free the old blocks, allocate new blocks, log to WAL, write the block, commit the WAL entry, then update metadata (size and block pointers).

Libraries and tools: C, SQLite 3.46.1 amalgamation (embedded, no external dependency), standard C file I/O, Makefile, MinGW GCC. All calls are local function calls; there is no network protocol.

Tasks Completed (7 pts)

Main tasks implemented, based on the supplied source archive and README.

Task Completed / Evidence	Team Member
Studied EXT2 block allocation and implemented the bitmap allocator (block 0) with alloc/free	Manoj Singh
Implemented raw block read/write/zero on the disk image	Manoj Singh
Designed the SQLite schema (files, wal_log), indexes and root-directory seeding	Vignesh Singh
Implemented metadata CRUD and path resolution	Vignesh Singh
Implemented WAL logging and commit, plus recover	Satvik Malasi
Built the CLI: init, mkdir, touch, write, cat, ls, tag, search, recover, info	Team
Implemented multi-block files (up to 12 blocks) and overwrite with block reuse	Team
Implemented tag-based search (key, or key and value)	Moulik Bohra
Embedded SQLite, wrote the Makefile, built hybridfs.exe	Moulik Bohra
Wrote demo.bat, test.py, test_runner.c, crash_sim.c and query_db.c (SQL prompt)	Satvik Malasi

Challenges/Roadblocks (7 pts)

Current risks and planned resolution.

1. **Broken WAL Recovery:** Redo code fails because uncommitted entries are currently removed, causing block writes to be lost. Additionally, metadata and bitmap updates occur outside the WAL. *Fix:* Use SQLite transactions and implement a proper redo/undo policy.
2. **State Persistence Issues:** The `init` command wipes existing images, resetting state between runs and causing directory errors. *Fix:* Add CLI arguments and prevent `init` from overwriting active images.
3. **Outdated Documentation and Tests:** The UTF-8 BOM issue is resolved in `main.c`, but the documentation and test scripts remain outdated.
4. **Missing `rm` Command:** Although `db_delete_file` exists, there is no CLI command to clean up allocated blocks or check whether directories are empty.
5. **Weak Integrity Checks:** Missing `UNIQUE` constraints on `(parent, name)` allow duplicate entries, and return values for disk and WAL operations are currently unchecked.
6. **Size and Buffer Limits:** Files are capped at 12 KB, CLI writes are limited to 2 KB, and the 8 MB bitmap capacity does not match the 10 MB disk image size.
7. **Unindexed Search and Phase 1 Gaps:** Tag searches rely on slow JSON `LIKE` scans with potential buffer truncation due to missing indexing. Additionally, Phase 1 deliverables (comprehensive tests and benchmark reports) remain incomplete.

Tasks Pending (7 pts)

Task Pending	
Implement rm (free blocks, handle non-empty directories)	Moulik Bohra
Real WAL recovery: redo/undo policy, transactions, recover-at-startup	Satvik Malasi
Persist image/DB paths; make init non-destructive	Vignesh Singh
`UNIQUE(parent_id, name)` and error handling on every write path	Satvik Malasi
Indirect blocks or larger file support; larger input buffer	Manoj Singh
Tags table or JSON1 with index; owner/date/size search commands	Vignesh Singh
Kill-mid-write crash test, plus a bitmap vs. metadata consistency checker	Team
Benchmark vs. a plain-file baseline; assertion-based tests (Linux and Windows)	Moulik Bohra
Document-management demo, final report, README cleanup	Team

Project Outcome/Deliverables (2 pts)

The expected outcome is a working prototype of a decoupled metadata and data storage system. It supports file create, read, update and search, with deletion planned. It has a relational metadata schema with indexes, a WAL-based consistency mechanism with a recovery command, a document-management demo showing tag search, and documentation comparing the design with EXT2, S3 and HDFS. The prototype exists today with all of the above except deletion, true crash recovery and benchmarks.

Progress Overview (2 pts)

Phase 1 Milestone	Status	Basis	Score
1. Metadata schema finalized, blob addressing designed	Done	`files` table with 12 direct block pointers and two indexes. SQLite replaced MySQL, and tags are a JSON column instead of a separate table.	100%
2. Data layer: blob write, read, delete	Mostly done	Write, read and block alloc/free work (multi-block file and overwrite tested). No delete command, so `rm` gives "Unknown command".	75%
3. Metadata layer: schema, CRUD, indexing	Mostly done	CRUD functions, path resolution and indexes work. No unique `(parent, name)` constraint, and the delete function isn't wired to the CLI.	85%
4. WAL, crash recovery, search interface	Half done	WAL logs and commits every block write, and tag search works. Recovery only deletes uncommitted entries and never replays them. Tag search is a full table scan, and there is no kill-mid-write test.	50%

Codebase Information (2 pts)

Repository link, branch, and important commits.

- **Repository:** https://github.com/mnu-11/Hybrid-Decoupled-Database-System.git
- **Source Modules:** `cli/main.c`; `fs/fs.c`; `db/db.c`; `wal/wal.c`; public headers in `include/`; embedded SQLite source under `sqlite/`.
- **Build System:** `Makefile` compiles all modules and links `hybridfs.exe` via MinGW GCC 64-bit.

Testing and Validation Status (2 pts)

Execution logs obtained from native Linux environment build.

Test	Status	Notes
Build	Pass	Compiles with warnings only.
`init` / `mkdir` / `touch` / `write` / `cat` / `ls`	Pass	Single session; 46-byte file read back correctly.
Multi-block file	Pass	1,500 bytes used blocks 1 and 2; bitmap matched.
Overwrite	Pass	Old content replaced correctly.
Tagging and search	Pass	By key and by key+value; a miss returned 0 results.
Empty file / missing path / wrong type	Pass	Clean error messages.
BOM input	Pass	Stripped correctly.
Cross-process use (`demo.bat` style)	Fail	Paths not persisted.
Duplicate names	Fail	Two `/d` entries created.
WAL recovery	Partial	Uncommitted entries removed, but not replayed.
`rm`	Fail	Not implemented.
Kill-mid-write test / Performance benchmark	Not done	Execution pending.

Deliverables Progress (2 pts)

- **Completed:** Data layer, metadata layer, WAL logging and commit, CLI, tag search, build system, demo scripts, and test scripts.
- **In Progress:** Recovery semantics, repeatable automated tests, and robust error handling.

- **Pending:** `rm` command, kill-mid-write crash testing, bitmap vs. metadata consistency checker, performance benchmarking, and final documentation cleanup.